

Projektdokumentation

FitFridge

Software Engineering

Media Systems SoSe 2026

Department Medientechnik der Fakultät DMI

Prof. Dr. Larissa Putzar, Sune Maute, Jörg Balzer

Gruppe 08

Sriraam Sinnarajah 2646750

Vu Thanh Tung Paeper 2761945

Inhaltsverzeichnis

1	Einleitung.....	3
1.1	Ausgangslage und Motivation.....	3
1.2	Zielsetzung und Anforderungen.....	3
1.3	Aufbau des Berichts.....	5
2	Grundlagen und Einordnung.....	5
2.1	Vergleichbare Anwendungen.....	5
2.2	Technologische Grundlagen.....	6
2.3	Schichtenarchitektur.....	6
2.4	OpenFoodFacts als externe Datenquelle.....	7
3	Projektkontext und Vorgehen.....	7
3.1	Vorgehensmodell und Aufgabenverteilung.....	7
3.2	Tech-Stack und Datenquellen.....	8
4	Entwurf und Implementierung.....	9
4.1	Systemarchitektur.....	9
4.2	Datenmodell.....	10
4.3	Nutzertrennung und Authentifizierung.....	11
4.4	Kühlschrankverwaltung und Nährwertberechnung.....	11
4.5	Produktsuche und Barcode-Erfassung.....	12
4.6	Mahlzeiten-Tracker.....	13
4.7	Fehlerbehandlung und Robustheit.....	14
5	Evaluation und Qualitätssicherung.....	15
5.1	Teststrategie.....	15
5.2	Testabdeckung.....	15
5.3	Grenzen der Evaluation.....	16
6	Diskussion.....	16
6.1	Bewertung der Architekturentscheidungen.....	16
6.2	Risiken und Verbesserungspotenziale.....	16
7	Fazit.....	17
7.1	Zielerreichung.....	17
7.2	Lessons Learned.....	17
	Literaturverzeichnis.....	18
	Abbildungsverzeichnis.....	18

1 Einleitung

1.1 Ausgangslage und Motivation

Ernährungs- und Fitness-Apps wie Yazio oder MyFitnessPal unterstützen Nutzerinnen und Nutzer beim Erfassen von Lebensmitteln, Kalorien und Makronährstoffen. Yazio bietet unter anderem Kalorienzählen, Nährstofftracking und Barcode-Scanning an.[1] MyFitnessPal beschreibt sich ebenfalls als Anwendung zum Tracken von Kalorien, Makros, Ernährung und Fitnesszielen.[2] Diese Anwendungen sind vor allem dann hilfreich, wenn Mahlzeiten bereits geplant oder gegessen wurden.

Im Alltag entsteht jedoch häufig eine andere Situation. Es sind noch bestimmte Kalorien und Makronährstoffe offen, gleichzeitig ist unklar, welche Lebensmittel überhaupt noch im Haushalt vorhanden sind und in welcher Menge. Klassische Tracking-Apps beantworten diese Frage nicht, weil sie protokollieren, was gegessen wurde, und nicht, was noch da ist. Genau an dieser Stelle setzt FitFridge an.

1.2 Zielsetzung und Anforderungen

FitFridge wurde als Webanwendung entwickelt, die einen digitalen Kühlschrank und einen Mahlzeiten-Tracker miteinander verbindet. Im Kühlschrank werden Lebensmittel mit Mengen, Einheiten und Nährwerten gespeichert, der Tracker verwaltet ein Tagesziel und stellt verbrauchte gegen verbleibende Werte. Beide Bereiche greifen ineinander: Wird eine Mahlzeit aus dem Kühlschrank geloggt, sinkt der Bestand automatisch, und bleibt von einem erfassten Produkt etwas übrig, wandert der Rest zurück in den Kühlschrank. Ziel war dabei eine mehrbenutzerfähige Anwendung, die beides in einer Oberfläche zusammenführt und mit möglichst wenigen Abhängigkeiten auskommt.

Das Projekt wurde modulübergreifend im Kontext der Module Software Engineering und Adaptive Systems and Artificial Intelligence umgesetzt. Der Software-Engineering-Teil umfasst eine einfache Authentifizierung für strikt getrennte Nutzerdaten, die Kühlschrankverwaltung, eine Produktsuche über OpenFoodFacts mit Barcode-Erfassung sowie den Mahlzeiten-Tracker. Dieser Bericht konzentriert sich auf diesen Teil. Die KI-gestützten Funktionen sind Gegenstand einer eigenen Dokumentation; der hier beschriebene Stand liegt als eigener Abgabezweig im Repository und ist ohne sie vollständig lauffähig.

Aus dieser Zielsetzung wurden vier Arbeitspakete, jedes mit einem Kriterium, an dem sich abnehmen ließ, ob es fertig ist.

Das erste ist die **Nutzerverwaltung** mit Registrierung, Anmeldung und Abmeldung. Fertig war sie, wenn der Aufruf einer geschützten Seite ohne Anmeldung auf der Anmeldeseite landet und ein Nutzer unter keinen Umständen Daten eines anderen zu sehen oder zu ändern bekommt.

Das zweite ist die **Kühlschrankverwaltung**: Anlegen aus einem Suchtreffer, Anzeige mit Live-Nährwerten, Ändern von Menge und Einheit, Löschen. Das Kriterium war doppelt: Die angezeigten Nährwerte müssen jederzeit zur gespeicherten Menge passen, und eine ungültige Eingabe darf weder etwas verändern noch einen Fehler produzieren.

Das dritte ist die **Produktsuche** über Text und Barcode, mit zusammengeführten lokalen und externen Treffern und optionaler Erfassung per Kamera. Entscheidend war, dass ein Ausfall der externen API eine leere Trefferliste ergibt und keinen Absturz.

Das vierte ist der **Mahlzeiten-Tracker** mit konfigurierbarem Tagesziel, Erfassen aus zwei Quellen, Bestandsabzug, Restübernahme und Kalender. Abgenommen wurde er daran, dass die Tagessummen mit der Summe der Einzeleinträge übereinstimmen und der Kühlschrankbestand konsistent bleibt.

Nicht-funktional standen die strikte Nutzertrennung auf Ebene jeder Datenbankabfrage, ein minimaler Technologie-Stack, Testbarkeit ohne Webserver und Internetverbindung sowie ein reproduzierbarer Demo-Zustand im Vordergrund.

1.3 Aufbau des Berichts

Kapitel 2 ordnet FitFridge in den Kontext vergleichbarer Anwendungen ein und erläutert die technischen Grundlagen sowie das verwendete Schichtenmodell. Kapitel 3 beschreibt Arbeitsweise, Aufgabenverteilung und Technologien. Kapitel 4 stellt die Umsetzung dar und bildet den Schwerpunkt. Kapitel 5 beschreibt Tests und Evaluation, Kapitel 6 diskutiert Architekturentscheidungen und Risiken, Kapitel 7 zieht Bilanz.

2 Grundlagen und Einordnung

2.1 Vergleichbare Anwendungen

Yazio und MyFitnessPal dienen als Vergleichsanwendungen, weil beide Apps ähnliche Grundfunktionen im Bereich Ernährungstracking anbieten. Dazu gehören Lebensmittelsuche, Kalorienzählen, Makrotracking und Barcode-Scanning.[1] [2] FitFridge übernimmt diese Grundidee des Trackings, erweitert sie jedoch um eine andere Perspektive: Die App berücksichtigt nicht nur, was gegessen wurde, sondern auch, welche Lebensmittel aktuell verfügbar sind.

Der zentrale Unterschied liegt in der Nutzung der gespeicherten Lebensmitteldaten. In klassischen Tracking-Apps werden Lebensmittel hauptsächlich dokumentiert; nach dem Erfassen einer Mahlzeit spielt der Eintrag keine Rolle mehr. In FitFridge ist derselbe Datensatz zugleich Bestandsposten. Produkte können unmittelbar aus dem Kühlschrankbestand zu einer Mahlzeit hinzugefügt werden, wobei der Bestand automatisch angepasst wird, und eine nicht verbrauchte Restmenge landet umgekehrt als neuer Posten im Kühlschrank. Dadurch entsteht ein stärker planender Ansatz.

FitFridge richtet sich vor allem an fitnessinteressierte Nutzerinnen und Nutzer, die Kalorien und Makronährstoffe bewusst steuern möchten, etwa im Rahmen von Muskelaufbau, Diät oder allgemein bewussterer Ernährung. Zusätzlich hilft der digitale Kühlschrank dabei, vorhandene Lebensmittel gezielter zu verwenden.

2.2 Technologische Grundlagen

FitFridge ist eine serverseitig gerenderte Webanwendung auf Basis von Flask, einem schlanken Web-Framework für Python, das keine Projektstruktur vorgibt und Erweiterungen optional lässt.[3] Für ein Projekt dieser Größe ist das ein Vorteil, weil es keine versteckte Konfiguration und keine automatisch erzeugten Dateien gibt: Was die Anwendung tut, steht vollständig im Quelltext.

Das Frontend besteht aus HTML-Vorlagen und etwas JavaScript, mehr nicht. Gefüllt werden die Vorlagen von Jinja2, der Vorlagensprache von Flask, die es erlaubt, eine Seite von einer anderen abzuleiten.[4] Alle fünf Seiten bauen deshalb auf derselben Grundvorlage auf, in der Navigation, Nutzerbereich und Hinweismeldungen ein einziges Mal stehen. Ein Frontend-Framework gibt es nicht, und es muss nichts umgewandelt werden, bevor eine Seite im Browser landet.

Als Datenbank dient SQLite, eine eingebettete, serverlose SQL-Datenbank, die alles in einer einzigen Datei ablegt.[5] Der passende Treiber gehört zu Python dazu, es braucht also weder eine zusätzliche Abhängigkeit noch einen separaten Datenbankprozess. Eine Bibliothek, die Datenbankzeilen automatisch in Python-Objekte übersetzt, kam bewusst nicht zum Einsatz; sämtliche Datenzugriffe sind handgeschriebenes, parametrisiertes SQL.

2.3 Schichtenarchitektur

Grundlage der gesamten Anwendung ist ein durchgängiges Schichtenmodell. Im Browser laufen Jinja2-Templates und Vanilla-JavaScript, darunter folgen Flask-Routen, Fachlogik-Services und Repositories mit reinem SQL auf einer SQLite-Datenbank. Abhängigkeiten verlaufen ausschließlich von oben nach unten.

Die Rollenverteilung ist eng gefasst, und das mit Absicht. Routen kümmern sich nur um HTTP: Formulardaten entgegennehmen, an einen Service delegieren, eine Rückmeldung setzen und eine Seite ausliefern. Services enthalten die Fachlogik, kennen aber keine Flask-Objekte und lassen sich deshalb ohne laufenden Webserver aufrufen. Repositories enthalten SQL und sonst nichts. Diese Architekturentscheidung trennt HTTP-Verarbeitung, Fachlogik und Datenzugriff voneinander und ist der Grund, warum alle zehn automatisierten Tests ohne laufenden Webserver auskommen.

2.4 OpenFoodFacts als externe Datenquelle

Produkt- und Nährwertdaten stammen von Open Food Facts, einer freien, kollaborativ gepflegten Lebensmitteldatenbank. Die API stellt unter anderem Informationen zu Produktnamen, Marken, Mengenangaben und Nährwerten bereit.[6] Genutzt werden zwei Endpunkte: einer liefert zu einem Barcode einen vollständigen Datensatz, der andere zu einem Suchbegriff eine Trefferliste, allerdings nur mit Barcode, Name und Marke.

Die Datenqualität ist heterogen. Energieangaben liegen mal in Kilokalorien und mal nur in Kilojoule vor, Mengenangaben sind Freitext und reichen von „400 g“ über „1.5 l“ bis „2 x 250 g“, einzelne Nährwertfelder fehlen ganz. Der Client muss deshalb normalisieren statt zu vertrauen; wie das umgesetzt ist, beschreibt Kapitel 4.5.

3 Projektkontext und Vorgehen

3.1 Vorgehensmodell und Aufgabenverteilung

Die Projektorganisation erfolgte nach einem Kanban-orientierten Vorgehen. Aufgaben wurden über Discord abgestimmt und nach Bearbeitungsstand strukturiert. Zusätzlich wurden Git-Feature-Branche genutzt, um einzelne Arbeitspakete getrennt voneinander zu entwickeln; die Integration in den Main-Branch erfolgte nach Absprache und gegenseitiger Prüfung über einen Merge-Branch. Abstimmungen fanden donnerstags statt, dazu kamen eine Zwischen- und eine Abschlusspräsentation. Über die Laufzeit von Ende April bis Ende Juli 2026 entstanden 94 Commits auf mehreren Branches.

Die Gruppe startete mit drei Mitgliedern. Im Verlauf zeigte sich, dass ein Mitglied nur in sehr geringem Umfang zum Ergebnis beitrug. Zugesagte Arbeitspakete wurden wiederholt nicht geliefert und mussten kurzfristig von den übrigen Mitgliedern übernommen werden, meistens genau dann, wenn die Zeit ohnehin knapp war. Nach mehreren erfolglosen Absprachen wurde das Mitglied aus der Gruppe entfernt und die Aufgaben wurden neu verteilt. Angenehm war das nicht, rückblickend aber richtig.

Die Projektarbeit wurde damit von Sriraam Sinnarajah und Vu Thanh Tung Paeper gemeinsam umgesetzt. Beide arbeiteten am Backend. Sriraam Sinnarajah bearbeitete schwerpunktmäßig die Fachlogik rund um den Mahlzeiten-Tracker sowie die fachliche Ausarbeitung. Vu Thanh Tung Paeper übernahm zusätzlich wesentliche Teile der API-Anbindung, der Datenbankstruktur, der OpenFoodFacts-Integration, der Frontend-Umsetzung und der finalen technischen Konsolidierung.

3.2 Tech-Stack und Datenquellen

FitFridge wurde als Webanwendung mit Python 3.10+ und Flask 3 umgesetzt. Für die Datenhaltung wird SQLite verwendet, das Frontend basiert auf Jinja2-Templates und Vanilla-JavaScript, externe Produktdaten kommen über OpenFoodFacts. Ergänzend wurde Claude (Anthropic) als Entwicklungsunterstützung in Bereichen eingesetzt, in denen Vorwissen fehlte, insbesondere bei der Frontend-Umsetzung und beim Debugging. Tabelle 1 fasst die Technologien und die Gründe für ihre Wahl zusammen.

Tabelle 1: Tech-Stack

Quelle: Eigene Darstellung

Schicht	Technologie	Warum
Backend	Python 3.10+, Flask 3	Minimalistisch, gibt keine Projektstruktur vor
Datenbank	SQLite über sqlalchemy	Eine Datei, kein Server, kein Migrationstool nötig
Frontend	Jinja2-Templates und Vanilla-JS	Kein Build-Tool, kein Framework
Externe Daten	OpenFoodFacts API über urllib	Freie Produktdatenbank, keine zusätzliche Abhängigkeit
Sicherheit	werkzeug.security, certifi	Passwort-Hashing beziehungsweise TLS-Zertifikate
Barcode	BarcodeDetector API, Fallback zxing-wasm	Erst die native Plattformfunktion, Bibliothek nur als Rückfallebene
Tests	pytest	Fixtures und monkeypatch ohne weiteres Zutun

Als Datenquelle im engeren Sinn kommt nur OpenFoodFacts von außen; alles andere entsteht in der Anwendung selbst. Die lokale Produkttabelle wächst mit jedem übernommenen Treffer und wird bei der nächsten Suche zuerst befragt, Mengen, Einheiten, Tagesziele und Mahlzeiten stammen aus den Formularen der Nutzer.

Die Datenbank wird bei jedem Serverstart frisch aufgesetzt: Das Schema verwirft alle Tabellen und legt sie neu an, anschließend wird ein Demo-Account mit sieben Kühlschrankprodukten, einem Tagesziel und drei Mahlzeiten eingespielt. Für eine Vorführung ist das genau richtig, weil jeder Start denselben Zustand erzeugt. Zur Laufzeit sind die Daten voll persistent, sie überleben nur keinen Neustart.

4 Entwurf und Implementierung

4.1 Systemarchitektur

Die Anwendung besteht aus elf Python-Modulen mit rund 1 380 Zeilen Code, die sich eindeutig den Schichten aus Kapitel 2.3 zuordnen lassen: `routes.py` bildet die HTTP-Schicht, die beiden `*_service.py` die Fachlogik, die drei `*_repo.py` den Datenzugriff. Dazu kommen ein Modul für Verbindung und Schema, eines für die Nährwertumrechnung, eines als Adapter zur externen API und eines für die Demo-Daten (siehe Abb. 1).

[Abbildung 1 einfügen]

Liegt bereits vor als `abb1_schichtenmodell.png`: fünf Ebenen mit Pfeilen ausschließlich nach unten, je Ebene die beteiligten Module und ein Stichwort zur Aufgabe, rechts die beiden Querschnittsmodule und der externe Dienst.

Abbildung 1: Schichtenmodell

Quelle: Eigene Darstellung

Das Schichtenmodell zeigt allerdings nur die Soll-Ordnung. Welche Module tatsächlich voneinander abhängen, dokumentiert das Klassendiagramm in Abbildung 2. Darin werden drei Abkürzungen sichtbar, die im Schichtenmodell verborgen bleiben: `routes.py` spricht für die Produktsuche direkt mit dem Repository und dem OpenFoodFacts-Adapter, statt einen Service dazwischenzusetzen, und `meal_tracker_service.py` greift sowohl auf `fridge_service.py` als auch auf dessen Repository zu, weil das Loggen einer Mahlzeit den Kühlschrankbestand verändert. Die erste Abkürzung ist Bequemlichkeit, die zweite folgt aus der Fachlichkeit.

[Abbildung 2 einfügen]

Liegt bereits vor als `abb2_klassendiagramm.png`: Module mit ihren Funktionen und «uses»-Abhängigkeiten, schichtkonforme Kanten grau, Abkürzungen farbig hervorgehoben.

Abbildung 2: UML-Klassendiagramm der Module

Quelle: Eigene Darstellung

Zwei Entwurfsregeln halten das Modell stabil. Erstens greift keine Route direkt auf ein Repository zu, sobald eine fachliche Entscheidung dahintersteckt; reine Lesezugriffe ohne Logik dürfen dagegen direkt bedient werden. Zweitens rechnet kein Repository: Nährwerte werden ausschließlich in einem einzigen Modul umgerechnet, das weder Datenbank noch Framework kennt.

Der Datenbankzugriff ist an den Anwendungskontext gebunden. Die Verbindung wird beim ersten Zugriff geöffnet, für alle weiteren Zugriffe derselben Anfrage wiederverwendet und am Ende automatisch geschlossen. Ein Request nutzt damit genau eine Verbindung, und Tests können sich einen eigenen Kontext mit einer temporären Datenbankdatei aufmachen.

4.2 Datenmodell

Das Schema besteht aus fünf Tabellen. Die zentrale Entwurfsentscheidung betrifft die Nährwerte: In `product` stehen ausschließlich Referenzwerte pro 100 g, Gesamtnährwerte eines Kühlschranks werden nie gespeichert (siehe Tabelle 2 und Abb. 3).

Tabelle 2: Tabellen des Datenmodells

Quelle: Eigene Darstellung

Tabelle	Inhalt	Besonderheit
user	Benutzername, Passwort-Hash	Benutzername ist eindeutig
product	Name, Marke, Barcode, vier Nährwerte pro 100 g	Barcode ist eindeutig und dient als fachlicher Schlüssel
fridge_item	Bestandsposten mit Menge, Einheit, Gewicht pro Stück	verweist auf Nutzer und Produkt
meal_tracker_settings	Tagesziel und Makroverteilung in Prozent	ein Datensatz je Nutzer
meal_tracker_entry	geloggte Mahlzeit mit absoluten Nährwerten	Zeitstempel mit Standardwert „jetzt“

[Abbildung 3 einfügen]

Liegt bereits vor als `abb3_datenschema.png`: Entity-Relationship-Diagramm der fünf Tabellen mit Kardinalitäten und farblich abgesetzten Nährwertspalten.

Abbildung 3: Datenmodell

Quelle: Eigene Darstellung

Dass Produkt und Bestandsposten getrennt sind, erlaubt es, dasselbe Produkt mehrfach und für mehrere Nutzer zu führen, ohne die Stammdaten zu duplizieren. Kauft ein zweiter Nutzer dieselbe Packung, entsteht ein zweiter Bestandsposten, aber kein zweiter Produktdatensatz.

Bei den Mahlzeiteinträgen läuft es bewusst andersherum: Dort stehen absolute Nährwerte, keine Referenzwerte. Ein Tagebucheintrag ist eine Aussage über etwas, das bereits passiert ist. Würde er auf den Produktdatensatz zeigen, könnte eine spätere Korrektur der Produktnährwerte rückwirkend die Vergangenheit verändern, und das darf nicht sein.

Das Gewicht pro Stück löst ein alltägliches Problem. Eier führt man sinnvoll in Stück, Nährwerte gibt es aber nur pro 100 g. Beim Anlegen lässt sich deshalb ein Stückgewicht hinterlegen, etwa 60 g für ein Ei. Fehlt es, liefert die Berechnung lieber Nullen als eine erfundene Zahl.

4.3 Nutzertrennung und Authentifizierung

Die Anmeldung selbst wurde nicht neu erfunden. Registrierung, Login, Logout und der Decorator, der geschützte Seiten absichert, folgen dem Aufbau des offiziellen Flask-Tutorials.[3] Passwörter landen als Prüfwert in der Datenbank, aus dem sich das Original nicht zurückrechnen lässt; das Verfahren bringt Werkzeug mit, die Bibliothek, auf der Flask aufsetzt.[7] Nach dem Anmelden merkt sich die Anwendung nur die Kennnummer des Nutzers in der Sitzung. Das ist bewährter Standard und war entsprechend schnell erledigt.

Die eigentliche Arbeit steckt in der Frage, was nach der Anmeldung passiert. Ein Login allein trennt noch keine Daten: Es sagt nur, wer jemand ist, nicht, worauf diese Person zugreifen darf. Genau hier lag die zentrale nicht-funktionale Anforderung, nämlich dass ein Nutzer unter keinen Umständen Daten eines anderen zu sehen oder zu verändern bekommt.

Umgesetzt ist das nicht auf der Ebene der Seiten, sondern eine Schicht tiefer, in jeder einzelnen Datenbankabfrage. Jede Abfrage, die Nutzerdaten liest oder schreibt, führt die Kennnummer des angemeldeten Nutzers als zusätzliche Bedingung mit. Entscheidend ist dabei, dass beim Schreiben nicht zuerst geprüft und dann geändert wird, sondern beides in derselben Anweisung steckt. Zwischen Prüfung und Änderung entsteht dadurch keine Lücke, in die sich etwas hineinschieben könnte, und die Regel kann nicht versehentlich vergessen werden, weil sie Teil der Abfrage selbst ist.

Praktisch heißt das: Wer in einem Formular eine fremde Nummer einträgt, läuft ins Leere. Die Abfrage findet nichts, es wird nichts geändert, und die Oberfläche meldet zurück, dass die Aktion nicht möglich war. Beim Löschen eines Kühlschranks kommt eine Prüfung davor, ob der Posten überhaupt dem angemeldeten Nutzer gehört; andernfalls antwortet die Anwendung, dass es ihn nicht gibt. Listen wiederum werden gar nicht erst ungefiltert geladen. Geschützt sind auf Seitenebene der Mahlzeiten-Tracker, das Hinzufügen von Produkten und das Löschen; die Kühlschrank-Startseite ist absichtlich auch ohne Anmeldung erreichbar, bleibt dann aber leer.

Eine Ausnahme soll nicht unter den Tisch fallen. Kühlschranks, die zu gar keinem Nutzer gehören, werden weiterhin gefunden. Diese Regel stammt aus einer frühen Projektphase und greift nur bei Datenbeständen aus jener Zeit; heute bekommt jeder neue Posten von Anfang an einen Besitzer. Sauber ist es trotzdem nicht, deshalb taucht der Punkt in Kapitel 6.2 noch einmal auf.

4.4 Kühlschrankverwaltung und Nährwertberechnung

Die Kühlschrankübersicht ist die Startseite. Sie lädt die Posten des Nutzers und ergänzt jeden um seine berechneten Nährwerte, anschließend summiert sie über alle Posten hinweg Kalorien, Protein, Kohlenhydrate und Fett. Die drei Makronährstoffe erscheinen als Donut-Diagramme, die Kalorien als Gesamtwert daneben (siehe Abb. 4).

[Abbildung 4 einfügen]

Zu zeigen: Startseite nach Anmeldung mit dem Demo-Account, sichtbar die drei Donuts, der Kalorien-Gesamtwert und mehrere Posten mit Menge und gerechneten Nährwerten, idealerweise einer davon in Stück.

Abbildung 4: Kühlschrank-Übersicht mit Live-Nährwerten

Quelle: Eigene Darstellung

Die Umrechnung selbst ist unspektakulär, und das ist gut so. Jede Einheit wird über eine Faktortabelle auf Gramm zurückgeführt, Volumen dabei vereinfachend als 1 ml gleich 1 g behandelt, danach werden die Referenzwerte mit dem Verhältnis zur Bezugsgröße von 100 g multipliziert.

Interessanter sind die Sonderfälle, denn die sind als Nullwerte modelliert und nicht als Ausnahmen. Eine Menge kleiner oder gleich null, eine unbekannte Einheit oder eine Stück-Angabe ohne hinterlegtes Stückgewicht ergeben ein Ergebnis aus lauter Nullen. Für die Oberfläche heißt das: Eine ungültige Eingabe verändert nichts und erzeugt keine Fehlerseite. Die aufrufenden Stellen bleiben dadurch frei von Ausnahmebehandlung, was eine Menge Sonderfälle gespart hat.

Ein Detail beim Ändern einer Menge ist wichtig: Wird keine Einheit mitgegeben, ändert sich nur die Menge, Einheit und Stückgewicht bleiben unangetastet. Genau dieser Weg wird beim Abzug durch den Mahlzeiten-Tracker verwendet, damit sechs Eier nach dem Verbrauch von zweien nicht plötzlich in Gramm geführt werden.

4.5 Produktsuche und Barcode-Erfassung

Die Suche ist die Schnittstelle zur externen Datenquelle und deshalb die Stelle mit dem meisten Normalisierungsaufwand. Beide Suchwege liegen hinter einer einzigen Eingabe. Besteht sie ausschließlich aus Ziffern, wird sie als Barcode behandelt und direkt aufgelöst. Andernfalls läuft eine Textsuche in zwei Stufen: zuerst die lokale Datenbank, danach die externe Suche. Beide Trefferlisten werden über den Barcode dedupliziert. Zuerst lokal zu suchen funktioniert ohne Netzzugriff und bevorzugt Produkte, die im Projekt schon verwendet wurden.

Die Sortierung der externen Treffer folgt einem einfachen Punktesystem: Eine exakte Übereinstimmung mit dem Suchbegriff bringt 60 Punkte, ein enthaltener Suchbegriff 35, ein enthaltenes Einzelwort 15, alles andere null. Vorher werden Akzente und Sonderzeichen entfernt, sodass „Müsli“ und „Muesli“ zusammenfallen. Bei gleichem Punktestand entscheidet der Proteingehalt, danach der niedrigere Kaloriengehalt. Das ist bewusst auf die Zielgruppe zugeschnitten: Wer nach „Joghurt“ sucht, bekommt eher die proteinreiche Variante nach oben gespült als die Sahnevariante.

Zwei weitere Funktionen glätten die Datenqualität. Die eine prüft die möglichen Energiefelder in fester Reihenfolge und rechnet notfalls Kilojoule in Kilokalorien um. Die andere liest aus dem Freitextfeld der Packungsangabe eine Menge samt Einheit heraus, erkennt Multiplikatoren wie „2 x 250 g“ und normalisiert Milligramm, Kilogramm, Liter, Zentiliter und Deziliter auf Gramm beziehungsweise Milliliter (siehe Abb. 5).

[Abbildung 5 einfügen]

Zu zeigen: Suchseite nach einer Textsuche mit mehreren Treffern samt Name, Marke und Nährwerten pro 100 g.

Abbildung 5: Produktsuche mit Trefferliste

Quelle: Eigene Darstellung

Die Anbindung ist bewusst ausfalltolerant. Netz- und Parserfehler werden abgefangen und zu einer leeren Trefferliste; scheitert das Nachladen eines einzelnen Treffers, fällt nur dieser weg. Ist Open Food Facts nicht erreichbar, sieht der Nutzer also nichts statt eines Absturzes. Die gesamte externe Abhängigkeit steckt in genau einem Modul, was sich beim Testen als Glücksfall herausgestellt hat.

Ergänzend lässt sich der Barcode mit der Kamera erfassen. Zuerst wird die native Barcode-Erkennung des Browsers genutzt, die ohne zusätzliche Bibliothek auskommt.[8] Ist sie nicht verfügbar, etwa in Firefox oder auf iOS, wird ein WebAssembly-Decoder nachgeladen.[9] Für diesen Fall wird das Kamerabild vorbehandelt, also beschnitten, in Graustufen gewandelt und im Kontrast angehoben. Das klingt aufwendiger, als es ist, macht aber den Unterschied zwischen „erkennt nichts“ und „erkennt sofort“, gerade bei glänzenden Verpackungen (siehe Abb. 6). Den vollständigen Ablauf vom Suchbegriff bis zum fertigen Kühlschrankposten zeigt Abbildung 7.

[Abbildung 6 einfügen]

Zu zeigen: geöffnete Kamera-Ansicht mit Zielrahmen über einem echten Barcode.

Abbildung 6: Barcode-Erfassung über die Kamera

Quelle: Eigene Darstellung

[Abbildung 7 einfügen]

Liegt bereits vor als `abb7_sequenzdiagramm.png`: Sequenzdiagramm über lokale Suche, externe Suche, Auswahl, Wiederverwendung oder Neuanlage des Produkts und Anlegen des Bestandspostens.

Abbildung 7: Sequenzdiagramm Produktsuche und Kühlschrank-Aufnahme

Quelle: Eigene Darstellung

4.6 Mahlzeiten-Tracker

Der Tracker führt Tagesziel und tatsächlichen Verzehr zusammen. Das Ziel besteht aus einem

Kalorienwert und einer prozentualen Verteilung auf Protein, Kohlenhydrate und Fett, eingestellt über Regler (siehe Abb. 8).

[Abbildung 8 einfügen]

Zu zeigen: Tracker-Seite mit mehreren erfassten Mahlzeiten, Tagesziel und der Gegenüberstellung von verbraucht und übrig.

Abbildung 8: Mahlzeiten-Tracker mit Tagesübersicht

Quelle: Eigene Darstellung

Damit die Verteilung immer aufgeht, werden die drei Werte auf eine Summe von 100 Prozent normalisiert. Negative Eingaben werden auf null gezogen, eine Gesamtsumme von null fällt auf die Standardverteilung 30/40/30 zurück, und ein durch Rundung entstehender Rest von wenigen Zehnteln wird auf den Fettanteil aufgeschlagen. Aus Zielkalorien und Prozentwerten ergeben sich die Zielmengen in Gramm über die Brennwerte von 4 kcal je Gramm Protein und Kohlenhydrate sowie 9 kcal je Gramm Fett; dem werden die Tagessummen gegenübergestellt.

Das Erfassen einer Mahlzeit ist als Warenkorb umgesetzt. Nutzer sammeln nacheinander mehrere Positionen, aus dem eigenen Kühlschrank oder aus einem neu gesuchten Produkt, und verbuchen den Korb anschließend in einem Schritt. Kommt eine Position aus dem Kühlschrank, wird die verbrauchte Menge vom Bestand abgezogen, wobei die Restmenge bei null abgeschnitten wird, sonst stünden irgendwann minus 30 g Joghurt im Kühlschrank. Kommt sie aus einem neu gesuchten Produkt und ist eine Restmenge angegeben, etwa weil von 200 g nur 120 g gegessen wurden, entsteht für die verbleibenden 80 g ein neuer Kühlschrankposten. Der Warenkorb ist damit die Stelle, an der Verzehr und Bestandsführung zusammenlaufen (siehe Abb. 9).

[Abbildung 9 einfügen]

Zu zeigen: Hinzufügen-Dialog mit gefülltem Warenkorb, beiden Reitern und dem Feld für die Restmenge.

Abbildung 9: Warenkorb und Restübernahme in den Kühlschrank

Quelle: Eigene Darstellung

Eine nachträgliche Mengenkorrektur skaliert die gespeicherten Nährwerte proportional; Mengen kleiner oder gleich null werden abgewiesen. Für die Rückschau enthält der Tracker eine Kalenderansicht. Das Monatsraster stammt aus der Standardbibliothek, welche Tage bereits Einträge haben, ermittelt eine einzelne Datenbankabfrage; diese Tage werden markiert (siehe Abb. 10). Der gewählte Tag bleibt in der Sitzung hängen, damit die Auswahl das Abschicken eines Formulars übersteht. Das war einer dieser Fehler, die man erst im echten Betrieb bemerkt: Vorher sprang die Ansicht nach jeder Aktion zurück auf heute.

[Abbildung 10 einfügen]

Zu zeigen: aufgeklapptes Monatsraster mit mehreren markierten Tagen und hervorgehobenem ausgewählten Tag.

Abbildung 10: Kalenderansicht mit markierten Tagen

Quelle: Eigene Darstellung

4.7 Fehlerbehandlung und Robustheit

Ein Prototyp darf Lücken haben, abstürzen darf er nicht. Durch die Anwendung zieht sich deshalb ein Muster, das in 21 Fehlerbehandlungen umgesetzt ist, elf davon allein in der HTTP-Schicht.

Ungültige Eingaben werden zu neutralen Werten statt zu Ausnahmen: Eine Menge kleiner oder gleich null liefert Nullen, ein nicht lesbarer Zahlenwert aus einem Formular wird zum Vorgabewert, ein manipuliertes Datum in der Adresszeile fällt still auf den heutigen Tag zurück. Ausfälle von außen werden zu leeren Ergebnissen, sodass Open Food Facts ausfallen kann, ohne dass FitFridge ausfällt. Und unzulässige Zugriffe laufen ins Leere, weil die Zugehörigkeit zum Nutzer in derselben Abfrage geprüft wird, in der geschrieben wird.

Dieselbe Haltung findet sich im Browser: Der Barcode-Scanner darf einzelne Kamerabilder nicht auswerten können, ohne abzurechnen, und wenn die Kamera gar nicht erst geöffnet werden kann, erscheint ein Hinweis in verständlicher Sprache statt einer Meldung aus der Browserkonsole.

Vollständig ist das nicht. Für unerwartete Ausnahmen gibt es keine eigene Fehlerseite, dort greift die Standardausgabe von Flask. Für den Prototyp ist das vertretbar, für einen produktiven Einsatz wäre es der erste Nachtrag.

5 Evaluation und Qualitätssicherung

5.1 Teststrategie

Die Qualitätssicherung basiert auf automatisierten Tests mit pytest und manueller Erprobung im Browser. Externe Dienste wie OpenFoodFacts werden in den Tests ersetzt. Dadurch laufen die Tests offline und reproduzierbar; die Ergebnisse hängen nicht davon ab, ob eine externe API gerade antwortet.

Beides ergibt sich unmittelbar aus der Architektur. Da die Fachlogik in Services liegt und keine Flask-Objekte kennt, lassen sich die Funktionen direkt aufrufen. Jeder Test erzeugt eine eigene Anwendungsinstanz mit einer temporären Datenbankdatei und legt das Schema frisch an. Die Demo-Daten bleiben im Testmodus bewusst außen vor, damit jeder Test von einem leeren und vorhersagbaren Zustand ausgeht.

5.2 Testabdeckung

Der Abgabestand enthält zehn automatisierte Tests in drei Dateien, aufgeschlüsselt in Tabelle 3.

Tabelle 3: Testdateien und Anzahl

Quelle: Eigene Darstellung

Testdatei	Anzahl	Schwerpunkt
test_meal_tracker.py	6	Tagesziele, Tagessummen, Bestandsabzug, Nutzertrennung, Mengenskalierung, Restübernahme
test_nutrition_integration.py	2	Nährwertberechnung aus Referenzwerten und Menge
test_api_db.py	2	Datenbank-Roundtrip und Auswertung der OpenFoodFacts-Antwort

Getestet wurde gezielt dort, wo ein Fehler entweder still falsche Zahlen erzeugt oder Nutzerdaten vermischt. Die Tests belegen unter anderem, dass eine Makroverteilung von 20/30/10 korrekt auf 33,3/50,0/16,7 normalisiert wird, dass ein Verbrauch von 100 g den Bestand von 400 g auf 300 g senkt, dass der Versuch, einen fremden Eintrag zu löschen, fehlschlägt, während der Eigentümer denselben Eintrag problemlos löschen kann, und dass von 200 g Mango nach 120 g Verzehr exakt 80 g als neuer Kühlschrankposten übrig bleiben.

5.3 Grenzen der Evaluation

Getestet sind Services und Repositories, nicht die Routen. End-to-End-Tests gibt es nicht, damit ist nichts automatisiert abgesichert, was Anmeldeablauf, Formularverarbeitung oder Weiterleitungen betrifft. Der Barcode-Scanner fällt ebenfalls heraus, weil er Kamerazugriff voraussetzt; er wurde

manuell auf mehreren Geräten und in mehreren Browsern erprobt. Eine Nutzerstudie fand nicht statt; was zur Bedienbarkeit gesagt werden kann, stammt aus der eigenen Erprobung und dem Feedback nach den Präsentationen.

6 Diskussion

6.1 Bewertung der Architekturentscheidungen

Die zentrale Entscheidung war der Verzicht auf zusätzliche Werkzeuge: kein Frontend-Framework, kein Build-Tool, keine Zwischenschicht zwischen Python und der Datenbank, kein eigenes Werkzeug für Schemaänderungen. Für den Projektrahmen hat sich das bewährt. Das Projekt startet mit zwei Befehlen, es gibt keine generierten Artefakte, und jede Abfrage steht als Text im Quelltext.

Kostenlos ist das nicht. Handgeschriebenes SQL bedeutet Wiederholung, und weil zwischen Datenbank und Code keine übersetzende Schicht liegt, fällt ein Tippfehler in einem Spaltennamen erst zur Laufzeit auf. Weil Änderungen am Schema nicht schrittweise nachgezogen werden, kostet außerdem jede Anpassung der Tabellen die vorhandenen Daten. Die Schichtentrennung dagegen hat sich uneingeschränkt gelohnt, und ihr Nutzen wurde erst beim Testen richtig sichtbar.

6.2 Risiken und Verbesserungspotenziale

Das größte Risiko ist das Demo-Setup. Die Datenbank wird bei jedem Serverstart neu aufgebaut, was für den Prototyp ausreicht, weil dadurch ein reproduzierbarer Testzustand entsteht. Für ein reales Produkt wäre diese Lösung nicht geeignet; dort müsste eine persistente Datenbank eingesetzt werden, bei der Änderungen an der Struktur über Migrationen kontrolliert durchgeführt werden.

Auf der Sicherheitsseite bleibt einiges offen. Der geheime Schlüssel für die Sitzungen steht fest im Code und gehört in eine Umgebungsvariable, die Formulare haben keinen CSRF-Schutz, der Anmeldevorgang ist nicht gegen wiederholtes Durchprobieren begrenzt, und die in Kapitel 4.3 genannten besitzerlosen Altposten gehören per Migration bereinigt.

Fachlich bleiben zwei Vereinfachungen stehen: Volumen wird durchgehend als 1 ml gleich 1 g gerechnet, was bei Wasser passt, bei Öl aber rund zehn Prozent danebenliegt, und nicht alle Formularfelder werden vor dem Speichern geprüft. Technisch sticht die Textsuche heraus: Weil die externe Such-API nur Barcodes liefert, muss für jeden Treffer ein weiterer Aufruf abgesetzt werden. Bei zehn Treffern sind das elf Anfragen, und das merkt man beim Warten deutlich.

7 Fazit

7.1 Zielerreichung

Die vier Arbeitspakete aus Kapitel 1.2 sind umgesetzt. Die Anwendung verwaltet Nutzer mit gehashten Passwörtern und trennt deren Daten auf Ebene der Datenbankabfragen, der Kühlschrank nimmt Produkte über Text- oder Barcode-Suche auf und zeigt Nährwerte je Posten und als Summe, der Tracker erfasst Mahlzeiten aus beiden Quellen, zieht Bestände ab, übernimmt Reste und zeigt die Tageswerte samt Kalender. Auch die nicht-funktionalen Ziele sind erreicht: Der Stack besteht aus Flask, Jinja2, SQLite und der Standardbibliothek, alle zehn Tests laufen offline und ohne Webserver, und der Demo-Zustand ist bei jedem Start reproduzierbar. Offen bleibt, was in Kapitel 6.2 steht.

7.2 Lessons Learned

Die wichtigste technische Erkenntnis war, dass sich der Nutzen einer sauberen Schichtentrennung erst

zeigt, wenn man etwas anderes tun will, als die Anwendung zu starten. Während der Entwicklung fühlte sich die Trennung zwischen Route, Service und Repository eine Weile nach zusätzlichem Aufwand für dasselbe Ergebnis an. Beim Schreiben der Tests kippte das schlagartig, weil kein einziger Kunstgriff nötig war, um die Fachlogik zu prüfen.

Die beste Entscheidung im Datenmodell war, ausschließlich Referenzwerte pro 100 g zu speichern und Gesamtwerte nie zu persistieren. Ein früher Zwischenstand hatte Gesamtnährwerte mitgeschrieben, und prompt zeigte die Übersicht nach jeder Mengenänderung etwas anderes an als die Detailansicht. Die Regel „rechnen statt speichern“ hat diese ganze Fehlerklasse auf einen Schlag beseitigt.

Ein weiterer Lernpunkt betrifft Abhängigkeiten. An mehreren Stellen reichte eine Plattform- oder Standardfunktion völlig aus, wo zuerst eine Bibliothek naheliegend schien. Zur Bibliothek gegriffen haben wir nur dort, wo die native Lösung nachweislich fehlte. Ebenso deutlich wurde, dass externe Dienste isoliert und ausfalltolerant angebunden werden müssen: Erst dadurch, dass der komplette Netzverkehr in einem Modul sitzt, das Fehler abfängt, wurde die Anwendung im Alltag benutzbar und im Test überhaupt prüfbar.

Organisatorisch zeigte das Projekt, dass regelmäßige Abstimmungen und eine klare Aufgabenstruktur wichtig sind. Am meisten gelernt haben wir dabei an der Stelle, die mit Code am wenigsten zu tun hat. Die Gruppe von drei auf zwei Personen zu verkleinern, war anstrengend und unangenehm; rückblickend haben wir zu lange darauf gehofft, dass zugesagte Arbeitspakete doch noch kommen, statt den Zustand früh anzusprechen. Dadurch stand in der Projektmitte Arbeit still, an der andere längst hätten weiterbauen können. Zwei Dinge nehmen wir mit: Arbeitspakete gehören von Anfang an genau einer Person und einem Termin zugeordnet, und ein ausbleibender Beitrag gehört beim nächsten Treffen auf den Tisch, nicht erst dann, wenn er den Fortschritt blockiert.

Literaturverzeichnis / Fußnoten

[1] Vgl. Yazio GmbH: Calorie Counter App, online verfügbar unter: <https://www.yazio.com/en/calorie-counter>, abgerufen am 30.07.2026.

[2] Vgl. MyFitnessPal: Calorie Tracker & Macro Tracking, online verfügbar unter: <https://www.myfitnesspal.com/>, abgerufen am 30.07.2026.

[3] Vgl. Pallets Projects: Flask Documentation, online verfügbar unter: <https://flask.palletsprojects.com/>, abgerufen am 30.07.2026.

[4] Vgl. Pallets Projects: Jinja Documentation, Template Inheritance, online verfügbar unter: <https://jinja.palletsprojects.com/en/stable/templates/>, abgerufen am 30.07.2026.

[5] Vgl. SQLite Consortium: About SQLite, online verfügbar unter: <https://www.sqlite.org/about.html>, abgerufen am 30.07.2026.

[6] Vgl. OpenFoodFacts: API Documentation, online verfügbar unter: <https://openfoodfacts.github.io/openfoodfacts-server/api/>, abgerufen am 30.07.2026.

[7] Vgl. Pallets Projects: Werkzeug, Security Helpers, online verfügbar unter: <https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>, abgerufen am 30.07.2026.

[8] Vgl. MDN Web Docs: Barcode Detection API, online verfügbar unter: https://developer.mozilla.org/en-US/docs/Web/API/Barcode_Detection_API, abgerufen am 30.07.2026.

[9] Vgl. zxing-wasm: ZXing-C++ WebAssembly Bindings, online verfügbar unter: <https://github.com/Sec-ant/zxing-wasm>, abgerufen am 30.07.2026.

Abbildungsverzeichnis

Abb. 1: Schichtenmodell.....	9
Abb. 2: UML-Klassendiagramm der Module.....	10
Abb. 3: Datenmodell.....	11
Abb. 4: Kühlschrank-Übersicht mit Live-Nährwerten.....	12
Abb. 5: Produktsuche mit Trefferliste.....	13
Abb. 6: Barcode-Erfassung über die Kamera.....	14
Abb. 7: Sequenzdiagramm Produktsuche und Kühlschrank-Aufnahme.....	14
Abb. 8: Mahlzeiten-Tracker mit Tagesübersicht.....	14
Abb. 9: Warenkorb und Restübernahme in den Kühlschrank.....	15
Abb. 10: Kalenderansicht mit markierten Tagen.....	15

Tabellenverzeichnis

Tabelle 1: Tech-Stack.....	8
Tabelle 2: Tabellen des Datenmodells.....	10
Tabelle 3: Testdateien und Anzahl.....	15